ARAZVERA TECHNOLOGIES — Level 3 Specialist Engineering Catalog

Client-facing evidence edition | 9 November 2026

Focus: PostgreSQL tenant isolation, secure backend reliability, recovery integrity, controlled releases and per-tenant SLO assessment.

Tier policy: “Level 3” is an internal ARAZVERA delivery tier, not a certification, an achieved billing rate or a claim of commercial production deployment. Our public case studies use synthetic data; proprietary source stays in private repositories.

A. Tenant security and PostgreSQL policy review

Customer problem: unintended cross-tenant access, privilege escalation or insecure row-level security configuration.

Scope: review authorized staging SQL/API boundaries, check least-privileged roles, reproduce negative paths, and deliver policy/permission findings with a PASS/HOLD recommendation.

Evidence today: Public PostgreSQL 16 RLS and restore lab has **14 real-database tests**, including forced RLS, different tenant logins and denied privilege escalation. The separate authorized FastAPI/PostgreSQL API lab has **14 tests**. These are not credentials for penetration testing or proof of deployed customer security audits.

B. Data recovery evidence and migration safety

Customer problem: uncertain backup/restore integrity, missing tenant records, policy changes and migration rollback risk.

Scope: isolated, authorized restore rehearsal; row and permission comparison; rollback checks; incident evidence and an English runbook.

Evidence today: The public lab tested an isolated restore into a second database on the **same server**. A new private Level 3 recovery evidence verifier compares canonical tenant data, role privileges, RLS policy declarations and visibility probes. **17 local tests and private GitHub Actions CI passed.** Its input is observed snapshots; different cluster IDs alone cannot establish an actual cross-host restore. That remains an explicit next acceptance milestone.

Additional tested Level 3 recovery lab: On 9 November 2026, a real PostgreSQL 16 custom-format backup was restored from one disposable cluster into a second, separately initialized cluster. Role provisioning, independent PostgreSQL system identifiers, restored RLS policy enforcement, tenant-login isolation, and canonical data equality all passed locally and in a separate private GitHub Actions integration job. Both containers were on **one physical computer**, with no network access or exposed ports. This does **not** prove recovery across physical servers, regions or production environments.

Additional verified failure-path evidence: A short PostgreSQL 16 pgbench lab processed 600 synthetic transaction attempts across two restricted tenant logins (12 concurrent clients in total). It verified uniqueness/counter consistency, tenant RLS separation, rollback after deliberately terminating one in-flight backend transaction and idempotent retry. The private GitHub Actions integration job passed. This does not establish production throughput, automatic failover or an independent physical-server recovery drill.

C. Webhook and API resilience

Customer problem: lost or duplicated callbacks, retries that exceed policy and untraceable partial failure.

Scope: signed ingress, idempotency contracts, bounded retry/dead-letter behavior, fault testing, reconciliation and audit-ready handover.

Evidence today: Public webhook lab has **16 tests**; transactional data reconciliation lab has **21 tests**. Real CRM vendors and production n8n runtime are outside this evidence unless separately integrated and approved.

D. Release safety and tenant-level SLO guardrails

Customer problem: high-risk schema/API changes with inadequate independent review, rollback evidence, canary measurement or tenant-level visibility.

Scope: evidence-based change gates, signed independent approval, staging rollback rehearsal, canary thresholds, per-tenant error/latency budget reviews and a release-blocker report.

Evidence today: Three private control modules - Release Guard, Per-Tenant SLO Guard and Signed Evidence Gate - passed **61 local unit tests** on Python 3.12, with private GitHub Actions passing on Python 3.11/3.12. Signed Evidence Gate binds reviewers, QA and client attestations to a canonical evidence digest using HMAC-SHA256 and prevents nonce replay with a local transactional SQLite store. The test identities, keys and observations are synthetic. No real IAM, live monitoring feed, production deployment or release approval is proven.

E. Transactional outbox, crashed workers and reliable event processing

Customer problem: multi-worker callback/event queues that lose work, duplicate sends after a crash, retain exhausted leases indefinitely, or lack an auditable dead-letter path.

Scope: design an authorized PostgreSQL-backed transactional outbox, replay-safe event IDs, atomic SKIP LOCKED claims, lease fencing, expiry recovery, bounded retry, dead-letter reconciliation, and fault-injection acceptance. Every external destination must separately support an idempotency key.

Evidence today: A new private PostgreSQL 16 kit passed **21 unit tests on Python 3.11/3.12 and a real Docker integration job** in GitHub Actions. The single-host lab used **8 concurrent Python workers**, processed **120 distinct initial synthetic events** without duplicate claims, then recovered an abandoned lease, rejected a stale acknowledgment,

exhausted a poison job, and safely dead-lettered two expired final-attempt leases. A separate business-state/outbox integration test verified both atomic COMMIT and atomic ROLLBACK on the caller-owned PostgreSQL transaction. Final accounting: **122 delivered jobs / 3 dead-letter jobs**. These are database-local receipts, **not** proof of exactly-once external API side effects, authenticated tenant identity, or production-scale performance.

F. Authenticated tenant API and operational queue monitoring

Customer problem: SaaS endpoints that rely on client-supplied tenant IDs, use overprivileged queue workers, or lack tenant-separated operational visibility.

Scoped deliverables: FastAPI authentication and authorization integration, restricted PostgreSQL roles and FORCE RLS, atomic business/outbox transactions, an independent queue worker, permission-controlled per-tenant queue health, process-restart checks, acceptance tests and an operational handover. Production IAM, monitoring alerting and deployment are separately scoped.

Verified evidence: The private integrated lab exercised three live local components: Uvicorn/FastAPI, PostgreSQL 16 Docker and an independently executed Python worker. It passed **15 synthetic integration acceptance checks**, including JWT signature/issuer/audience/expiry/scopes, different tenant database LOGINS, denied cross-tenant SQL reads/writes, restricted worker permissions, 18 database-local deliveries, atomic rollback, API restart durability, and queue metrics protected by a distinct ops:read scope. **All four private GitHub Actions jobs succeeded**, including the Python 3.11/3.12 unit-test matrix and both PostgreSQL Docker labs. No live customer IAM, public ingress, independent physical host, production SLA or external exactly-once API delivery is claimed.

G. Tenant-specific queue incident readiness and release risk evidence

Customer problem: stale queue health claims, silent dead letters, an overloaded tenant masking other tenants, and unchecked operational release decisions.

Scoped deliverables: risk and observability inventory, per-tenant queue health inputs, fail-closed HOLD/PASS staging evidence, tenant-specific abnormal-condition reproduction, recovery/runbook checklist and integration with customer-approved incident and release policies.

Verified lab: The private incident readiness module passed **47 local unit tests** and **19 end-to-end acceptance checks on three successive runs**. PostgreSQL FORCE RLS and JWT ops:read constrain queue observations by tenant. Real synthetic drills verified tenant A backlog or dead letter => tenant A HOLD, tenant B unaffected, and recovery after workers clear a backlog => staging observation PASS. All four relevant private CI jobs passed. The observed SHA256 digest does not sign telemetry; there is no real SLO, 24/7 alert transport, enterprise identity provider or approved automated failover.

H. Public-key JWT verification and enterprise identity hardening

Customer problem: signature algorithm confusion, stale signing keys, incorrect issuer/audience trust, single-tenant scope leakage and unpredictable identity-provider outages.

Scoped deliverables: identity threat-model and token contract review; issuer/audience/JWKS pinning; RSA signing-key rotation and revocation test matrix; strict claims and scope verification; tenant-to-database privilege separation; staged fail-closed outage drills; incident acceptance evidence and an English deployment handover. Connecting a real corporate OIDC provider, HTTPS, key management and operational support must be separately authorized and tested.

Verified synthetic evidence: A private opt-in **RS256/JWKS verification module** with a separate mock metadata/identity process was implemented. **30 new OIDC tests** increased the private outbox/incident suite to **77 passing unit tests** on both Python 3.11 and Python 3.12. A new **13-check live mock-JWKS / FastAPI / PostgreSQL 16 integration lab** passed: forged/expired/wrong issuer/wrong audience tokens rejected, HS256 downgrade blocked, rotated signing keys accepted and revoked keys rejected after refresh, JWKS outages failed closed, and the original restricted PostgreSQL tenant boundaries remained intact. The prior 19-check FastAPI/worker incident lab and original transactional outbox Docker test also passed in the **single successful sequential private GitHub Actions acceptance run** on 9 November 2026.

Explicit limit: The issuer implements only OIDC-compatible metadata and public JWKS, **not** a complete OIDC authorization-code/PKCE sign-in flow, corporate IAM, HTTPS production identity service, customer access control, multi-host recovery or security certification. The public-key cache has a bounded revocation delay. A successful synthetic lab cannot be sold as a fully integrated enterprise SSO deployment.

I. Network-path fault injection and transactional recovery

Customer problem: an API or background worker silently accepts incomplete work during a database transport outage, retries a failed operation unsafely, or leaves the business transaction and event queue inconsistent.

Scoped deliverables: authorized staging fault matrix, synthetic transport-path interruption, bounded failure responses, atomic business/outbox rollback checks, safe replay/reconciliation evidence, tenant-level incident readiness, runbook and handover. Multi-host failover and RTO/RPO must be scoped and demonstrated separately.

Verified lab: A disposable PostgreSQL 16 container, FastAPI and independent Worker used a private loopback TCP relay that can interrupt existing synthetic SQL connections and refuse new ones. **Nine acceptance assertions passed on four local executions and in the single successful private GitHub Actions acceptance gate**. The lab verified HTTP 503 and no phantom events during an outage, termination of an in-flight uncommitted business/outbox transaction, atomic rollback of both records, recovery without PostgreSQL restart, safe reprocessing after reconnection and restoration of staging queue readiness.

Boundary: This is **not** a real cross-host network partition, physical node shutdown, production database failover or enterprise RTO/RPO guarantee. The laboratory did not alter firewall settings, customer credentials or live infrastructure. Deploying such tests to client environments requires written consent, safety limits, monitoring, recovery plans and separate

customer approval.

J. Tenant incident monitoring and durable alert delivery

Customer problem: a SaaS tenant's operational incident is silently missed, a brief transient condition floods the on-call team, or a downstream notification service loses an acknowledgement and generates duplicate escalation.

Scoped deliverables: tenant-aware queue observation and incident criteria, RLS-scoped read permissions, incident state and debounce rules, durable alert outbox, receiver idempotency and retry contracts, staging fault drills, recovery notifications, diagnostic report and an incident escalation runbook. A 24/7 deployment, real incident paging provider and reliable on-call scheduling must be separately scoped and accepted.

Verified private laboratory: A one-shot synthetic monitor polled signed JWT-protected FastAPI queue readiness for two different FORCE RLS PostgreSQL tenants. **107 unit tests passed** across the private suite. A real **11-check FastAPI/PostgreSQL/Worker/SQLite alert journal and separate HTTP receipt-receiver integration** passed three consecutive local runs and the complete sequential private GitHub Actions acceptance gate. The synthetic scenario opened a tenant-only backlog incident, preserved an alert after an ambiguous HTTP 503 acknowledgement, deduplicated a retry at the receiver, persisted the journal across a fresh Python process, closed the incident after worker recovery, opened a new incident for a dead letter, and delivered after a restarted receiver. Final receiver accounting held three unique receipts and no unsent alerts.

Boundary: This is one-shot localhost monitoring, **not** a 24/7 installed alert service, a managed alert provider integration, an uptime/SLO measurement, on-call escalation, production disaster recovery or HA replicated SQLite solution. No external notification was sent. At-least-once sender delivery relies on a cooperative receiver's idempotency for deduplication.

Verification register — dated 9 November 2026

Asset	Verified	Scope limit
Public portfolio	**9 synthetic cases / 75 tests**	Python 3.11/3.12 CI and disposable PostgreSQL 16 where needed
Level 3 PostgreSQL RLS evidence	**14 of the 75 public tests**	Same-server recovery laboratory
Private Level 3 recovery verifier	**17 passing local tests** and passing CI	Not a real independent-server restore
Separate PostgreSQL cluster restore lab	**Local LAB_PASS + private Docker integration CI success**	Two real clusters on the same physical host; no external disaster recovery
Concurrent tenant and crash-recovery lab	**600 synthetic transaction attempts; rollback, replay and tenant RLS PASS in local Docker and private CI**	Up to 12 benchmark clients on one host; not a production-scale load or node-failure test
Private Level 3 release/SLO/signed evidence modules	**61 passing local tests** and passing Python 3.11/3.12 CI	HMAC test keys and offline observations; no live IAM or deployment
Private PostgreSQL transactional outbox	**21 unit tests + 12 Docker integration checks; 8 workers, 120 distinct initial events; same-transaction business COMMIT/ROLLBACK; private CI successful**	Single-host lab, no external provider delivery or independently authenticated DB tenant role
Authenticated FastAPI + tenant RLS + worker integration	**15 local integration checks + 4 private CI jobs passed**	Lab-issued JWTs, two synthetic tenant roles, one physical host, no external delivery or production identity provider
Per-tenant queue incident readiness	**47 local tests + 19 end-to-end checks repeated three times; all four private CI jobs passed**	Queue-specific lab control only, not real SLO or production release authorization
Private pinned RS256/JWKS token verification	**77 unit tests on Python 3.11/3.12 + 13 new live identity tests; full sequential private CI PASS**	Mock issuer, loopback HTTP and synthetic tenant roles; not a complete enterprise OIDC/SSO deployment
Simulated TCP transport failure and DB rollback	**9 synthetic checks passed in 4 local runs and in the final private sequential CI**	One physical host and localhost proxy; no real cross-host partition, HA failover or guaranteed RTO/RPO
Tenant incident monitoring and durable alert outbox	**107 private unit tests + 11-check end-to-end lab on three runs, complete sequential CI PASS**	One-shot synthetic polling; SQLite on one host and localhost mock receiver; not 24/7 or externally integrated alerting
Existing private Level 2 accelerators	**25 families / 313 recorded local tests**	Separate nonpublic baseline, not publicly reviewable

These counts refer to different suites; they are not customer deployments. No claim of SOC 2 accreditation, guaranteed uptime, signed penetration test or proven cross-region recovery is made.

Commercial positioning and engagement method

Proposed quotation ladder (not achieved income): Level 1 targets **$30–$80/h** for bounded implementation, Level 2 now targets **$100–$149/h** for complex backend and integrations, and **Level 3 targets $150–$250+/h** for specialist architecture, security boundaries, recovery engineering, incident readiness and accountable risk controls. These are proposed quote ranges, not verified market prices, historical contracts or guaranteed earnings.

Illustrative Level 3 fixed-scope proposals: architecture/risk discovery **$3,000–$7,500**, tenant-boundary and reliability hardening **$9,000–$25,000**, and controlled recovery/change assurance **$18,000–$60,000**, subject to scope, client infrastructure, authorized testing, delivery proof, and separately priced third-party costs. Larger engagements must not be sold as already production-validated. Quote milestones and deliverables in writing, not just a model rate.

Each engagement requires written scope and acceptance criteria, lawful client access, customer-owned credentials, controlled staging, negative-path tests, evidence capture, and independent client approval before any production change.

Public résumé and evidence

- Level 3 Specialist Résumé — PDF
- Level 3 Specialist Résumé — DOCX
- Level 3 Readiness and Limitations
- Engineering Portfolio

Contact: contact.arazvera@gmail.com

ARAZVERA TECHNOLOGIES | Evidence first. Private source remains private.